

地下城探险与任务规划系统项目报告

一、项目概述

本项目实现了基于 Web 的地下城探险与任务规划系统，采用前后端分离架构。玩家在随机生成的 20 个房间的地下城中探索、战斗、收集宝物，最终击败 Boss。系统综合运用图、树、队列、栈、列表等数据结构，实现完整的游戏逻辑和可视化交互。

核心特性： 自动地图生成、智能路径规划、树形战利品管理、D20 骰子战斗、Boss 动态移动、游戏状态管理、地图快照

二、运行说明

2.1 环境要求

- Python 3.10（需安装 Flask、flask-cors）
- 现代浏览器（支持 ES6 Module）
- 操作系统：Ubuntu 22.04 LTS

建议： 优先使用 **Linux 开发与运行**。在其它环境上可能会遇到字体、DPI/缩放、滚动条与表单控件样式差异、路径/换行符差异以及文件系统性能问题；这些差异会影响界面布局与快照一致性。

2.2 启动步骤

下面提供**运行环境搭建方式**，使用 Python 内置的虚拟环境。示例命令以 Linux 为准（请在 `dungeon_project` 目录下运行以下命令）。

使用 `virtualenv`（推荐在 Linux 环境下开发）：

```
# 创建并激活虚拟环境（Linux）
python3 -m venv .venv
source .venv/bin/activate

# 安装 Python 依赖
pip install -r requirements.txt
```

启动后端服务器：

```
python3 backend/app.py
```

访问应用：在浏览器打开 `http://localhost:5000`

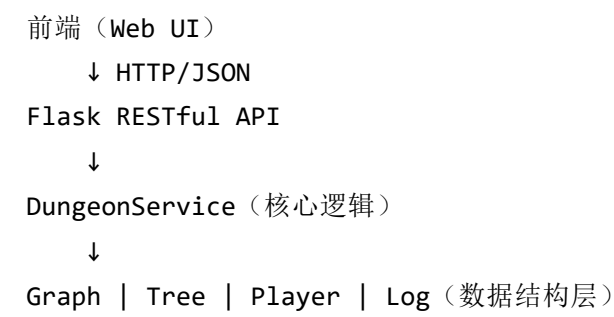
重要说明：

- 推荐 Python 版本：**3.10.x**（环境文件中指定为 **3.10.12**）。

三、项目结构

```
dungeon_project/
├── backend/                                # Python Flask 后端
│   ├── app.py                            # RESTful API 路由
│   ├── dungeon_service.py                # 核心业务逻辑
│   ├── graph.py                          # 图结构（邻接表、BFS/DFS）
│   ├── tree.py                           # 树结构（战利品分类）
│   ├── models.py                         # 数据模型（Room、Monster、Player等）
│   └── adventure_log.py                   # 日志列表管理
├── frontend/                             # JavaScript 前端
│   ├── index.html                        # 主界面（地牢主题UI）
│   └── static/js/
│       ├── main.js                       # 主控制器
│       ├── api_client.js                 # API调用封装
│       ├── map_renderer.js               # 地图渲染（vis-network）
│       ├── path_controller.js            # 路径搜索控制
│       ├── treasure_viewer.js            # 战利品展示
│       ├── log_viewer.js                 # 日志展示
│       ├── snapshot.js                   # 快照捕获与下载（html2canvas）
│       └── logger.js                     # 前端可开关日志封装
└── requirements.txt                       # 依赖清单
```

3.1 系统架构



四、数据结构设计与应用

4.1 图结构 (Graph) - 地图表示

实现文件： backend/graph.py

应用场景：

- 1. 地图生成时建立房间的连通关系
- 2. BFS 最短路径搜索
- 3. DFS 所有可行路径搜索
- 4. Boss 随机移动时获取相邻房间

4.2 树结构 (Tree) - 战利品管理

实现文件： backend/tree.py

应用场景：

- 1. 战利品分类存储和管理
- 2. 统计各类别宝物数量和总价值
- 3. 前端树形可视化展示

4.3 队列 (Queue) - BFS 最短路径

实现位置： backend/graph.py 的 find_shortest_path()

应用场景：

- 1. 玩家快速到达目标房间
- 2. 路径规划面板的"最短路径"功能

4.4 栈（Stack） - DFS 路径搜索

实现位置： backend/graph.py 的 find_all_paths()
应用场景：

- 1. 寻找多条可行路径供玩家选择
- 2. 路径规划面板的"所有路径"功能（默认最多返回前 5 条）

4.5 列表（List） - 冒险日志

实现文件： backend/adventure_log.py
应用场景：

- 1. 记录所有游戏事件（移动、战斗结果、宝物获取）
- 2. 前端日志面板实时展示
- 3. 游戏复盘和调试

五、基本功能实现（50 分）

5.1 地图生成与展示（10 分）

后端通过图结构生成随机地下城：为每个房间随机分配怪物、宝物，同时保证地图有解。使用 DFS 确保所有房间连通，Boss 房间至少有一条通往起点的路径。前端使用 vis-network 库渲染力导向布局，不同颜色节点区分房间类型，边表示连通关系。房间上会标注怪兽宝箱等信息（#后面标注的是房间号）。



玩家点击相邻房间触发移动。后端验证目标房间是否为当前房间的邻居，合法则更新玩家位置。移动到新房间触发事件：遭遇怪物自动战斗、有宝物则自动收集并添加到树结构。前端实时更新玩家状态面

板、日志列表。



5.2 战利品信息管理（10 分）

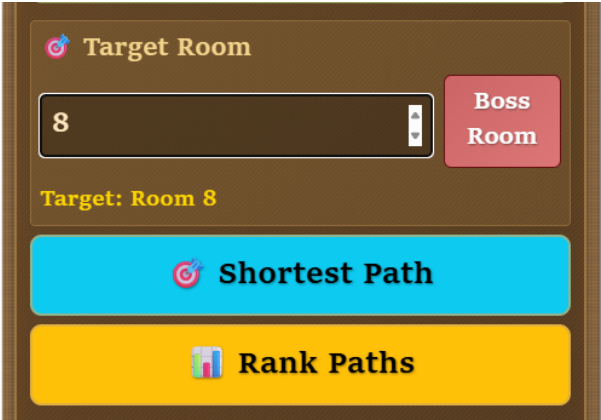
自动收集房间宝物，加入玩家背包和树结构。树结构按 5 大类别分类管理，每个宝物记录名称和价值。前端左侧"Treasure Collection"部分进行展示，统计总价值和各类别数量。点击每个物品左侧星标可将宝物收藏到收藏夹，可以单独进入收藏目录，进行收藏物品展示。

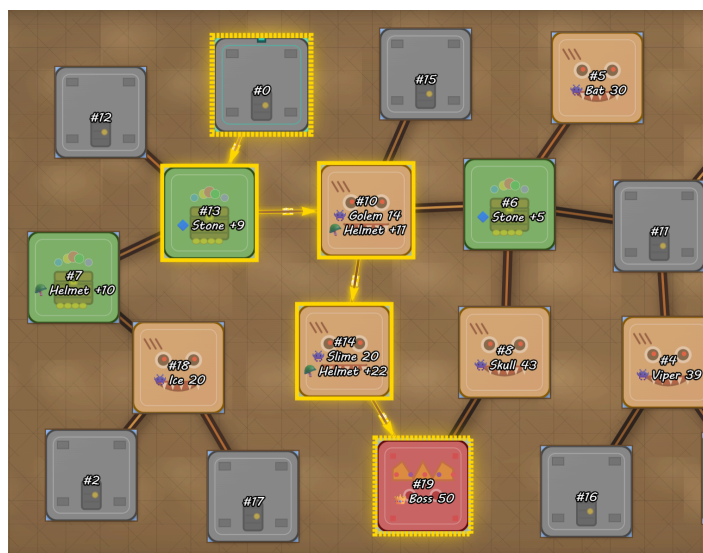




5.3 最短路径规划功能（10 分）

前端支持输入目标房间 ID（初始默认为 boss 房间），BFS 返回可行的最短路径（房间数最少）。路径以节点序列形式展示，在地图上高亮显示。该功能帮助玩家规划最优路线，避免危险房间。





5.4 多路径评估排序（10 分）

DFS 返回所有可行路径。路径按宝物总价值排序，总价值相同的路径仅展示其中的最短路径，前端最多显示前 5 条路径（若总数不到 5 条，则全部显示），供玩家权衡选择。支持灵活绕道收取宝藏，路径到达目标房间后不再继续探索统计。

战利品价值排行榜 (共7条路径, 显示前5名)

说明: 显示战利品总价值排名前5的路线 (按价值从高到低排序)。点击每条卡片的 **高亮此路径** 按钮在地图上高亮对应路线; 每次地图上最多高亮一条路径。

路径 1

价值: 89

路径: 0 → 13 → 7 → 13 → 10 → 6 → 11 → 4 → 1 → 4 → 11 → 6 → 10 → 14 → 19

长度: 14 步 | 武力提升: +89 | 总价值: 89

高亮此路径

路径 2

价值: 77

路径: 0 → 13 → 7 → 13 → 10 → 6 → 10 → 14 → 19

长度: 8 步 | 武力提升: +77 | 总价值: 77

关闭

Top Paths

1

0 → 13 → 7 → 13 → 10 → 6 → 11 → 4 → 1 → 4 → 11 → 6 → 10 → 14 → 19

89 | 14 steps | +89

2

0 → 13 → 7 → 13 → 10 → 6 → 10 → 14 → 19

77 | 8 steps | +77

Treasure Collection

The diagram illustrates a treasure map with 20 numbered locations (0-19) connected by paths. The locations are represented by icons: #0 (Stone), #1 (Stone), #2 (Helmet), #3 (Rat), #4 (Stone), #5 (Stone), #6 (Rat), #7 (Helmet), #8 (Rat), #9 (Rat), #10 (Golem), #11 (Golem), #12 (Helmet), #13 (Ice), #14 (Golem), #15 (Rat), #16 (Rat), #17 (Rat), #18 (Rat), #19 (Rat). The map shows a complex network of paths connecting these locations, with some locations highlighted in yellow.





Adventure Log



Stats

Total Adventures: 1

Moves: 0

Value: 68

会话 #0 进行中

2025/12/23 06:59:25



起点: 房间 0 →  当前: 房间 2



路径长度: 8 步 (9 房间)



当前武力: 78 (+68)



总收益: 68

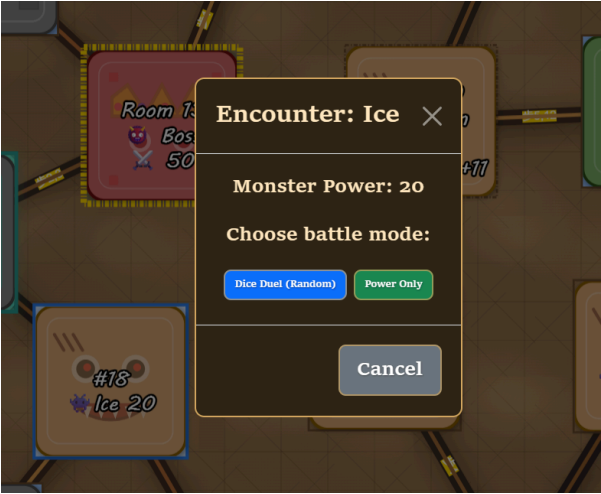


击败怪物: 0 只

六、拓展功能实现（40+分）

6.1 战斗模拟系统（10 分）

用户进入有怪兽的房间时可以选择直接比较武力值大小或者开启 D20 骰子对决模式。



D20 骰子对决机制：

战斗公式：

$$\begin{aligned} \text{玩家总战力} &= \text{玩家基础武力值} + \text{D20骰子点数} \\ \text{怪物总战力} &= \text{怪物基础武力值} + \text{D20骰子点数} \end{aligned}$$

战斗结果：玩家总战力 > 怪物总战力 → 胜利

特殊机制：

- 1. **暴击（Critical Hit）**：骰子=20，双倍伤害（武力值翻倍）
- 2. **大失败（Fumble）**：骰子=1，直接战斗失败
- 3. **Boss 战斗**：与 boss 对决时，允许筛两次骰子，取最大值

战斗流程动画：

- 1. 显示双方基础战力
- 2. 骰子滚动动画（1.5 秒）
- 3. 显示骰子点数和总战力
- 4. 战斗结果反馈



战斗奖励：

- 胜利：玩家可以获得额外武力值（基础 1-3 点，暴击额外奖励更多）
- 失败：扣除 2 点武力值
- 击败 Boss：游戏胜利



6.2 Web 可视化系统（10 分）

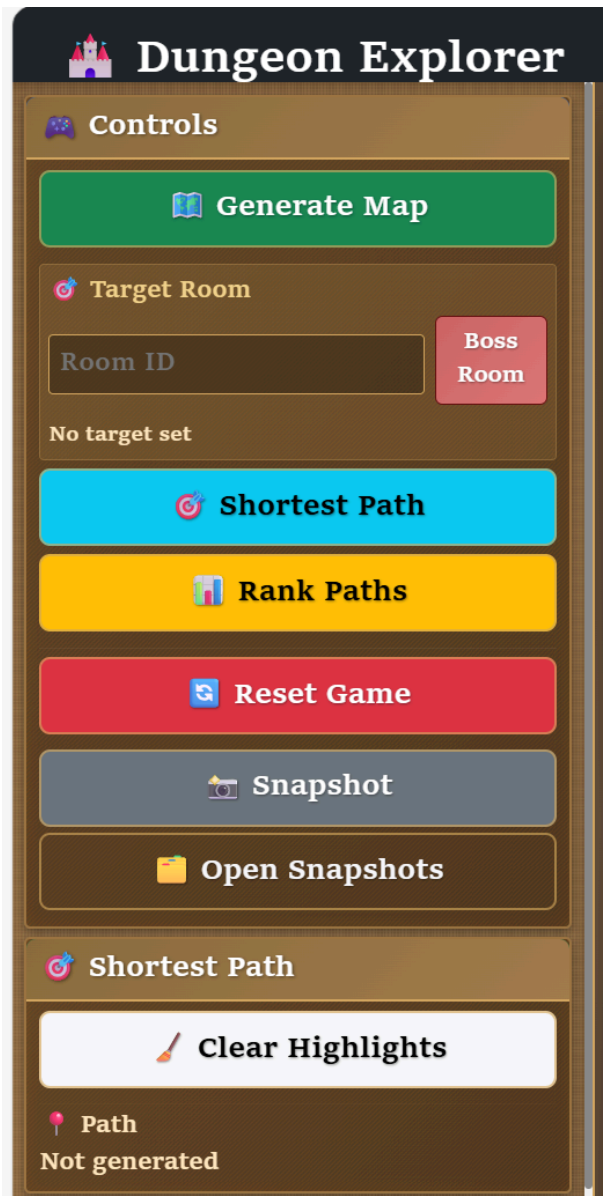
技术栈：

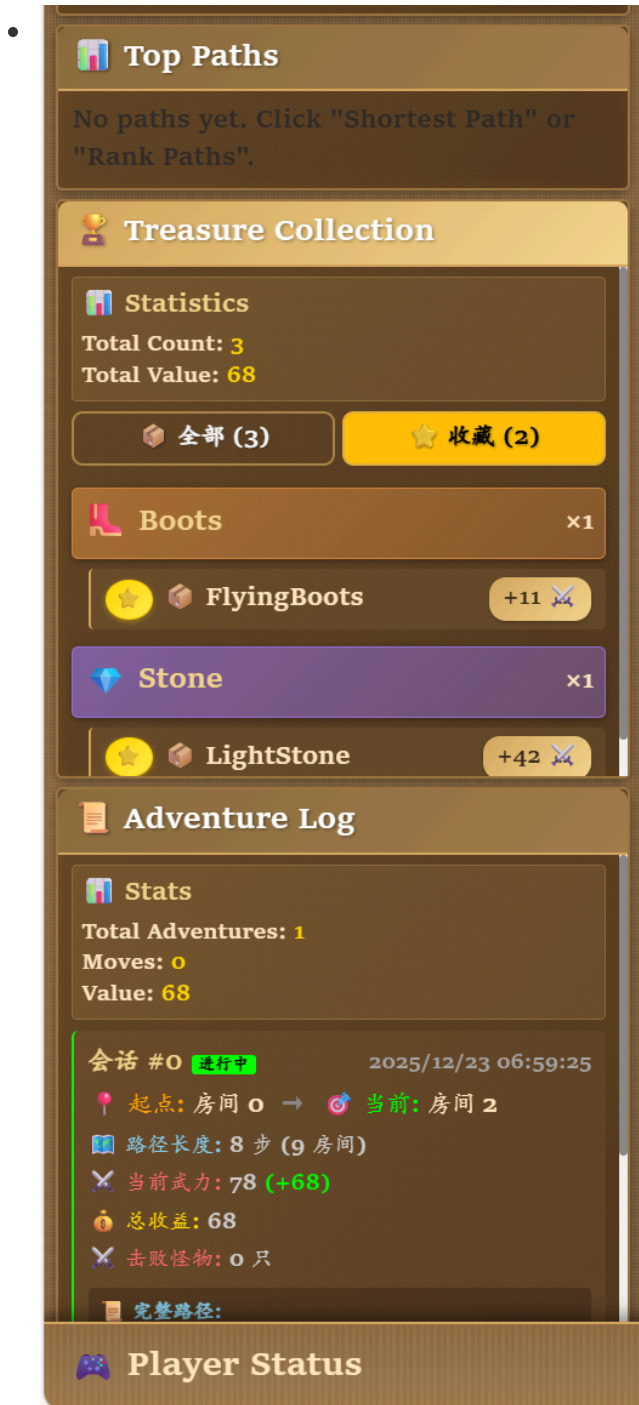
- 前端：HTML5 + CSS3 + JavaScript (ES6 Modules)
- 后端：Python Flask + RESTful API

- 可视化库：vis-network (图可视化)
- UI 框架：Bootstrap 5

界面设计：

1. 左侧控制面板（可上下滑动）：
- 游戏控制（生成地图、重置）
 - 目标房间设置
 - 路径搜索功能
 - 最短路径显示
 - 多路径排名列表
 - 地图快照与浏览下载
 - 战利品收藏展示
 - 探险日志记录
 -





2. 右侧地图区域：

- 动态交互式地图
- 点击节点触发移动
- 路径高亮

3. 顶部导航栏：

- 玩家实时状态（武力值、房间位置）
- 游戏标题和图标

视觉特效：

- 地牢风格深色主题

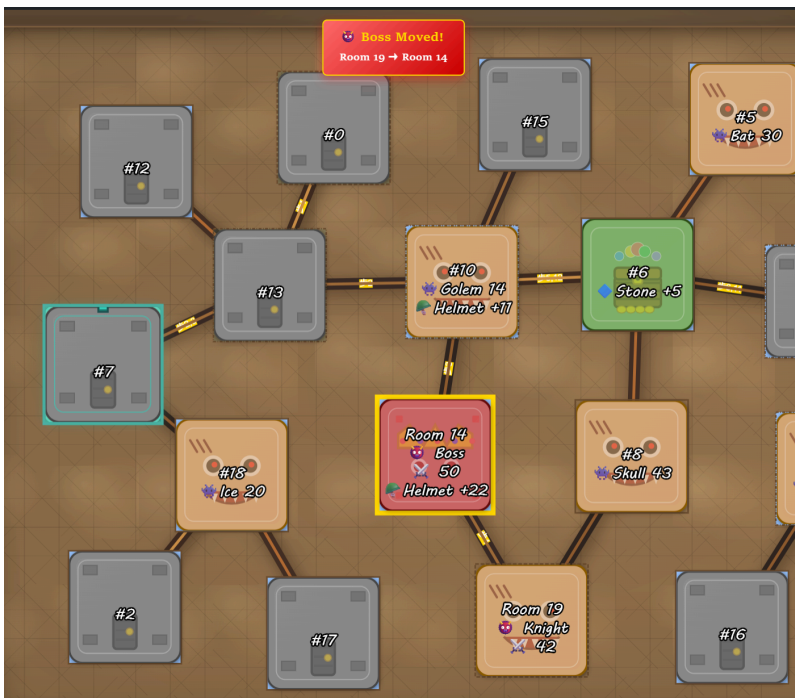
- 石质纹理背景
- 金色装饰边框

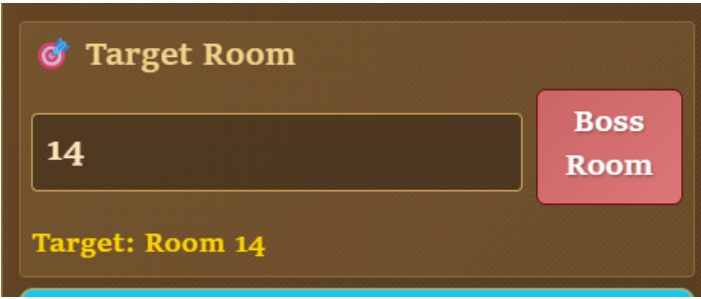


6.3 Boss 动态移动（10 分）

Boss 随机移动，并在移动时出现提醒。增加游戏难度和不确定性。前端实时更新 Boss 位置，探索最优路径的规划会实时规划;若玩家房间与 boss 房间相邻，boss 也有几率直接移动到玩家所在房间、触发战斗并判定游戏胜负。

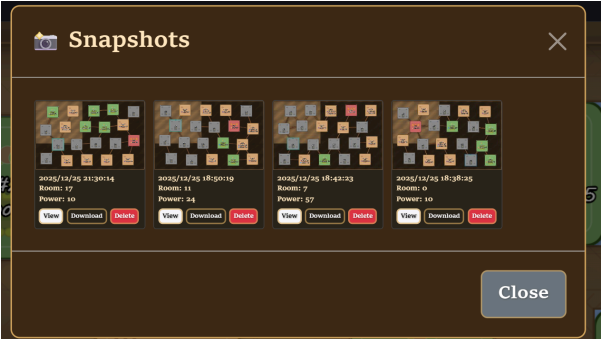
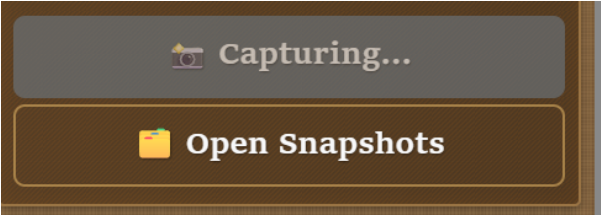
在 Boss 随机移动的短暂窗口内，允许玩家操作与 Boss 移动状态并发出现，同时触发“混乱战场”的视觉效果。即在短暂看到 boss 正确的移动信息后，boss 所在房间会呈现为普通房间、或者 boss 移动前后的房间均显示为 boss 房间等，此种情况可以进一步增加游戏难度，迫使玩家在决策时权衡即时响应与等待更新的风险。

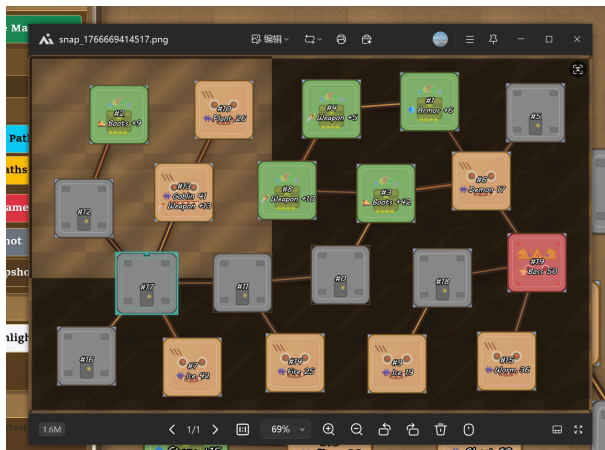
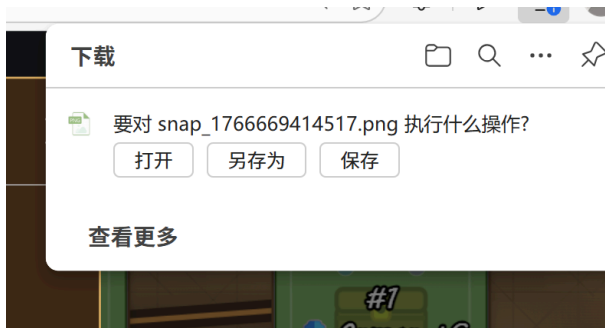




6.4 地图快照（10 分）

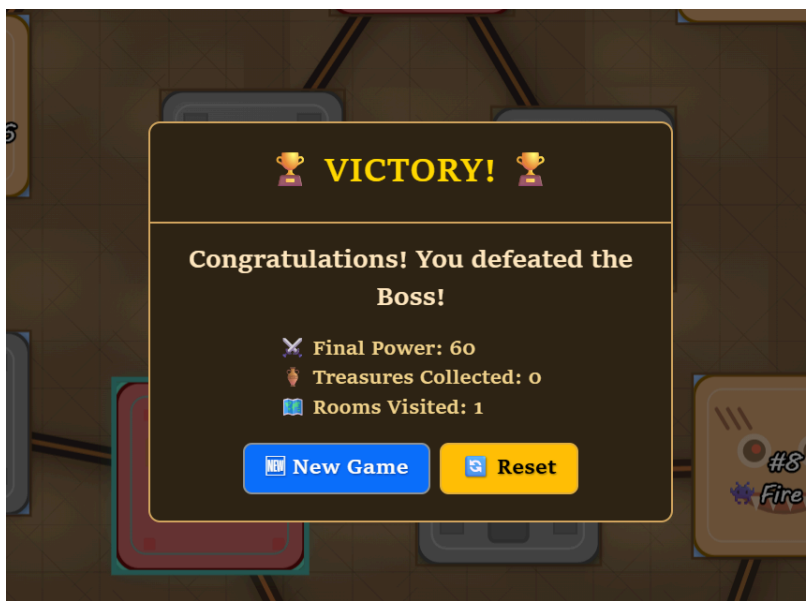
实现了页面级地图快照并记录当时的时间与玩家信息，支持历史管理、预览和本地下载。





6.5 游戏完整闭环（5-10 分）

当达到胜利或失败条件时展示结果（最终武力值、收集宝物数、访问房间数等），提供“开始新游戏”和“仅重置”两种操作；重置会调用后端 reset API 并清空前端状态与展示以等待玩家下一步操作，支持游戏完整闭环。





七、总结

本项目以数据结构与算法为核心，融合图、树、队列、栈等多种结构，打造了功能完善、交互流畅的地下城探险系统。前后端分离，模块清晰，算法与可视化紧密结合，极大提升了教学与演示效果。系统稳定，易扩展，创新性强，充分展现了数据结构在实际工程中的应用价值。

项目具备进一步智能化、联机化和可扩展潜力，可作为数据结构课程的优秀案例，支持教学、科研与工程实践推广。